TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN A DISTANCIA

TRABAJO FINAL INTEGRADOR

Andres Bonelli

Matias Carro

Sistema de gestión y trazabilidad

Control 107

INFORME TÉCNICO FINAL

Sistema web de gestión de turnos, ambulancias y control de insumos del servicio de emergencias 107

Universidad Tecnológica Nacional

Tecnicatura Universitaria en Programación a Distancia

Integrantes

- Matias Carro Legajo: 18743
- Andres Bonelli Legalo: 4824

Docente tutor: Sofia Raia

Fecha de entrega: Noviembre 2026

Resumen ejecutivo

El proyecto **Sistema de Gestión y Trazabilidad 107** surge de la necesidad de mejorar el registro y seguimiento de la información operativa asociada a las ambulancias del servicio de emergencias 107. El proceso relevado se apoya en planillas físicas para registrar el estado de los insumos de cada unidad, lo que motivó el desarrollo de una solución orientada a digitalizar estos controles y mantener su trazabilidad sin incrementar innecesariamente la carga operativa del personal.

Como solución se desarrolló una aplicación web que contempla la gestión de usuarios, ambulancias, categorías, insumos y turnos, junto con el registro de controles de ingreso y egreso, reposiciones, consumos, faltantes, observaciones e información histórica. El sistema diferencia las responsabilidades del administrador y del enfermero y contempla el tratamiento del oxígeno como un insumo crítico dentro del mismo modelo operativo.

La solución adopta una arquitectura en capas. El backend está desarrollado con Java y Spring Boot y expone una API REST; el frontend utiliza React, Vite y TypeScript como aplicación de página única, mientras que la persistencia se plantea mediante un modelo relacional con MySQL para producción.

La implementación obtenida permite centralizar los principales registros operativos definidos para el alcance del producto mínimo viable y mantener información histórica sobre turnos e insumos. De esta manera, el sistema busca aportar una base digital para mejorar la conservación, consulta y trazabilidad de la información operativa del servicio.

Link al video del proyecto:

https://www.youtube.com/watch?v=E8RFuTAf35E

Link al despliegue del Sistema:

https://control107.vercel.app

Resumen ejecutivo **1**
1. Introducción **1**
2. Descripción del problema y análisis de requerimientos **1**
2.1 Descripción del problema 1
2.2 Requerimientos funcionales 2
2.3 Requerimientos no funcionales 4
2.4 Historias de usuario 6
HU-01 – Crear Turno 6
HU-02 – Control de Ingreso 7
HU-03 – Control de Egreso 8
HU-04 – Consultar Historial del Turno 9
3. Marco teórico y tecnologías seleccionadas **10**
3.1 Marco teórico **10**
Arquitectura cliente-servidor y API REST 10
Arquitectura en capas y separación de responsabilidades 10
Aplicación de página única y modularidad del frontend 10
Modelo de datos relacional e integridad 11
Autenticación y autorización 11
3.2 Tecnologías seleccionadas y justificación 11
Java 17 12
Spring Boot 4.1.1 12
MySQL 8.0 12
React 19.3.1 + TypeScript 6.0.3 + Tailwind CSS 4.3.3 12
Vite 8.3.4 13
Docker / Docker Compose 13
Caddy 2 13
4. Descripción del proceso de desarrollo **14**
4.1 Metodología de desarrollo adoptada 14
4.2 Planificación y cronograma 14
Desvíos y replanificaciones 15
4.3 Narrativa del proceso de desarrollo 16
4.3.1 Etapa de relevamiento y análisis 16
4.3.2 Etapa de diseño 16
4.3.3 Etapas de implementación 16
4.3.4 Etapa de pruebas y validación 16
4.3.5 Etapa de despliegue 16
4.4 Control de versiones 17
Figura 1. Diagrama de Gantt **18**
5. Arquitectura y decisiones de diseño **19**
5.1 Diagrama de arquitectura general 19
5.2 Descripción de la arquitectura 19
5.3 Decisiones de diseño relevantes 20

ADR-01 – Arquitectura monolítica en capas.....20
ADR-02 – Persistencia mediante base de datos relacional.....20
ADR-03 – Simplificación del modelo operativo del dominio.....21
5.4 Modelo de datos.....21
5.5 Diagramas de secuencia o flujo.....22
5.5.1 Control de ingreso.....23
5.5.2 Control de egreso.....24
5.6 Diseño de la interfaz de usuario.....24
6. Implementación del sistema.....26
6.1 Estructura del proyecto.....26
6.2 Módulos o componentes principales.....27
6.2.1 Autenticación y seguridad.....27
6.2.2 Gestión de turnos.....27
6.2.3 Control de insumos.....27
6.2.4 Tratamiento del oxígeno.....28
6.2.5 Administración y resumen operativo.....29
6.3 Integración con sistemas externos.....29
7. Manual de usuario.....30
7.1 Introducción al manual.....30
7.2 Acceso al sistema.....31
7.3 Descripción de funcionalidades.....32
Administrador.....32
Enfermero.....32
7.4 Preguntas frecuentes (FAQ).....34
8. Manual de implementación (despliegue).....34
8.1 Requisitos del entorno.....34
8.2 Variables de entorno y configuración.....35
8.3 Procedimiento de instalación y despliegue.....35
8.4 Actualización del sistema.....36
8.5 Respaldo y recuperación.....36
8.6 Monitoreo y logs.....36
9. Pruebas y validación.....37
9.1 Estrategia de pruebas.....37
9.2 Casos de prueba.....38
9.3 Métricas de calidad.....38
9.4 Validación con usuarios.....39
10. Conclusiones.....39
10.1 Resultados obtenidos.....39
10.2 Lecciones aprendidas.....40
10.3 Limitaciones del sistema actual.....41
10.4 Trabajo futuro.....41
Referencias bibliográficas.....42

1. Introducción

El proyecto se desarrolla en el contexto del servicio de emergencias 107, donde parte de la información de las guardias se registra mediante planillas físicas. Estos registros incluyen datos sobre turnos, ambulancias, personal, disponibilidad de insumos, faltantes y observaciones. Esta modalidad dificulta conservar, consultar y relacionar posteriormente la información operativa.

El objetivo general es desarrollar una aplicación web que permita digitalizar y centralizar la gestión de turnos, ambulancias y controles de insumos. Como objetivos específicos se propone gestionar usuarios y sus responsabilidades, administrar las ambulancias y su dotación de insumos, registrar los controles realizados durante los turnos, detectar faltantes según los puntos de control y conservar información histórica de la actividad registrada.

El alcance comprende usuarios con roles de administrador y enfermero, ambulancias, categorías, insumos y turnos, junto con los controles de ingreso y egreso, reposiciones, consumos, faltantes y observaciones. También contempla el tratamiento del oxígeno como insumo crítico y la consulta de información histórica y estadísticas básicas. Quedan fuera del MVP la gestión de equipos variables de personal, las intervenciones clínicas detalladas, el control mecánico completo de las ambulancias y la comparación automática de cantidades entre turnos.

El informe presenta el problema y sus requerimientos, los fundamentos y tecnologías utilizadas, el proceso de desarrollo, la arquitectura, la implementación, el despliegue y las pruebas realizadas. Finalmente, se exponen los resultados y conclusiones del proyecto.

---

2. Descripción del problema y análisis de requerimientos

2.1 Descripción del problema

El uso de planillas físicas para registrar la información de las guardias dificulta mantener un historial centralizado de turnos, ambulancias y controles de insumos. La recuperación de antecedentes y el análisis de faltantes, consumos y observaciones requieren revisar registros distribuidos y realizar tareas manuales.

El control de insumos es uno de los puntos principales del proceso. Al comenzar una guardia se registran las cantidades disponibles y se identifican faltantes; durante el turno

pueden producirse reposiciones y consumos; y al finalizar se registran las cantidades restantes. Dentro de este control, el oxígeno requiere un tratamiento particular debido a su importancia operativa.

Las principales partes interesadas son el **enfermero**, responsable del control operativo durante la guardia, y el **administrador**, encargado de gestionar los recursos del sistema y consultar la información registrada. La falta de centralización afecta principalmente a estos usuarios, ya que dificulta el registro, la consulta de antecedentes y el seguimiento de faltantes y novedades. La centralización de la información busca facilitar estas tareas y mantener la trazabilidad de los controles realizados durante cada turno.

2.2 Requerimientos funcionales

Los requerimientos funcionales definen las funcionalidades que debe proporcionar el sistema. A continuación se presentan los requerimientos correspondientes al alcance definido, junto con su prioridad y criterio de aceptación.

Tabla 1. Requerimientos funcionales de Control107

ID	Descripción	Prior.	Criterio de aceptación
RF-01	**Gestión de Usuarios.** El sistema debe permitir crear, modificar, activar/desactivar y consultar usuarios con roles definidos.	Alta	Se pueden crear, consultar, modificar y activar o desactivar usuarios, manteniendo el rol correspondiente.
RF-02	**Gestión de Ambulancias.** El sistema debe permitir registrar ambulancias, modificar su disponibilidad y asignarlas a turnos.	Alta	Se pueden registrar y modificar ambulancias, gestionar su disponibilidad y asignarlas a un turno.
RF-03	**Gestión de Insumos y Categorías.** El sistema debe permitir crear insumos, asignarlos a categorías y definir puntos de control y criticidad.	Alta	Se pueden crear insumos, asociarlos a una categoría y establecer su punto de control y condición de criticidad.

ID	Descripción	Prior.	Criterio de aceptación
RF-04	**Programación de Turnos.** El administrador debe poder programar turnos indicando fecha, ambulancia asignada y franja horaria predeterminada (MAÑANA, TARDE o NOCHE).	Alta	El administrador puede crear un turno indicando fecha, ambulancia y una de las franjas horarias disponibles.
RF-05	**Inicio de Turno.** El enfermero debe registrar las cantidades encontradas de cada insumo al inicio del turno.	Alta	El enfermero puede registrar las cantidades encontradas de los insumos correspondientes al turno asignado.
RF-06	Control de Ingreso. El sistema debe calcular los faltantes comparando la cantidad encontrada con el punto de control mediante la fórmula **F = max(0, PC - CI).**	Alta	Al registrar el control de ingreso, el sistema calcula el faltante de cada insumo y registra cero cuando la cantidad encontrada es igual o superior al punto de control.
RF-07	**Reposición Durante el Turno.** El enfermero debe poder declarar reposiciones de insumos durante la guardia.	Alta	El enfermero puede registrar la cantidad repuesta de un insumo durante un turno en curso.
RF-08	**Control de Egreso.** El enfermero debe registrar las cantidades restantes al finalizar el turno.	Alta	El enfermero puede registrar las cantidades restantes de los insumos al finalizar el turno.
RF-09	**Cálculo de Consumo.** El sistema debe calcular el uso total del insumo mediante la fórmula **U_total = CI + CR - CE.**	Alta	El sistema calcula el consumo utilizando la cantidad de ingreso, la cantidad repuesta y la cantidad registrada al egreso.

ID	Descripción	Prior.	Criterio de aceptación
RF-10	**Cancelación de Turno.** El enfermero debe poder cancelar un turno por condiciones críticas y registrar el motivo.	Alta	El enfermero puede cancelar el turno y el motivo de la cancelación queda registrado.
RF-11	**Registro de Observaciones.** El sistema debe permitir registrar observaciones al ingreso y al egreso del turno.	Alta	Las observaciones registradas durante el ingreso y el egreso quedan asociadas al turno correspondiente.
RF-12	**Historial Operativo.** El sistema debe almacenar el historial de insumos utilizados por turno.	Alta	Al consultar un turno se puede acceder a la información registrada sobre los insumos utilizados durante la guardia.
RF-13	**Dashboard y Estadísticas.** El sistema debe mostrar estadísticas básicas de consumo y faltantes por ambulancia y por turno.	Media	El sistema permite consultar estadísticas básicas generadas a partir de los registros de consumo y faltantes.

2.3 Requerimientos no funcionales

Los requerimientos no funcionales establecen las características de calidad esperadas del sistema. Para el MVP se consideran aspectos de rendimiento, seguridad, disponibilidad, escalabilidad, usabilidad, mantenibilidad, trazabilidad e integridad de los datos.

Tabla 2. Requerimientos no funcionales de Control107

ID	Atributo	Descripción
RNF-01	Rendimiento	El sistema debe mantener tiempos de respuesta adecuados durante su utilización habitual. Se realizaron pruebas bajo

ID	Atributo	Descripción
		condiciones de carga normal para comprobar su funcionamiento, sin establecer para el MVP métricas específicas de carga pico.
RNF-02	Seguridad y autenticación	El sistema debe requerir autenticación para acceder a las funcionalidades protegidas y utilizar tokens JWT para mantener la identidad del usuario.
RNF-03	Control de acceso	El sistema debe restringir las funcionalidades y rutas disponibles de acuerdo con el rol del usuario. Los roles operativos definidos son Administrador y Enfermero.
RNF-04	Protección de datos	El sistema debe limitar la información almacenada a los datos necesarios para su funcionamiento operativo y evitar incorporar información clínica sensible fuera del alcance definido.
RNF-05	Disponibilidad	El sistema debe encontrarse accesible mediante infraestructura en línea.
RNF-06	Escalabilidad	La estructura del sistema debe permitir incorporar nuevas funcionalidades y ampliar el dominio sin requerir un rediseño completo. El backend organiza sus responsabilidades en componentes diferenciados y el frontend mantiene una estructura modular que facilita su extensión.
RNF-07	Usabilidad	La interfaz debe presentar información de forma clara mediante textos, colorimetría y mensajes de confirmación o error asociados a las acciones del usuario. La interfaz operativa del enfermero debe adaptarse a distintos tamaños de pantalla, siendo recomendable el uso de tablet para el trabajo móvil. Para las tareas administrativas se recomienda el uso de equipos de escritorio o notebooks por la cantidad y criticidad de la información gestionada.
RNF-08	Mantenibilidad	El código debe mantener separación de responsabilidades y una organización modular que facilite su corrección y extensión. El

ID	Atributo	Descripción
		backend separa configuración, controladores, servicios, repositorios, entidades y objetos de transferencia de datos, mientras que el frontend organiza la aplicación mediante componentes, páginas, servicios, hooks, interfaces, stores y utilidades.
RNF-09	Auditoría y trazabilidad	El sistema debe conservar la información necesaria para reconstruir la actividad operativa registrada en los turnos y mantener la trazabilidad de los controles de insumos.
RNF-10	Integridad de datos	El sistema debe preservar la consistencia de los datos y evitar registros incompatibles con las reglas del dominio definidas para turnos, usuarios, ambulancias e insumos.

2.4 Historias de usuario

Para representar los principales flujos funcionales del sistema se utilizan Historias de Usuario. Cada una identifica los actores involucrados, las condiciones necesarias para ejecutar la operación, el flujo principal, las situaciones alternativas o de excepción y el estado esperado al finalizar.

HU-01 – Crear Turno

Historia: Como administrador, quiero programar un turno indicando fecha, franja horaria, ambulancia y personal correspondiente, para que el enfermero asignado pueda realizar posteriormente el control operativo.

Actor: Administrador.

Precondiciones:

- El administrador debe estar autenticado.
- La ambulancia y el enfermero deben estar registrados en el sistema.
- La ambulancia y el enfermero deben encontrarse disponibles para la fecha y franja horaria seleccionadas.

Flujo principal:

1. El administrador selecciona la fecha y la franja horaria.
2. Selecciona la ambulancia.
3. Selecciona el enfermero.
4. Registra el nombre del chofer y del médico como datos informativos.
5. Confirma la creación del turno.
6. El sistema registra el turno en estado **PROGRAMADO**.

Flujos alternativos y de excepción:

- Si la ambulancia ya se encuentra asignada a otro turno en la misma fecha y franja horaria, el sistema rechaza la creación.
- Si el enfermero ya se encuentra asignado a otro turno en la misma fecha y franja horaria, el sistema rechaza la creación.
- Si alguno de los recursos requeridos no existe, el turno no se registra.

Postcondición: El turno queda registrado en estado **PROGRAMADO** y asociado al administrador, enfermero y ambulancia correspondientes.

HU-02 – Control de Ingreso

Historia: Como enfermero, quiero registrar las cantidades encontradas al inicio del turno, para detectar faltantes y dejar registrado el estado inicial de los insumos.

Actor: Enfermero.

Precondiciones:

- El enfermero debe estar autenticado.
- El turno debe estar asignado al enfermero.
- El turno debe encontrarse en estado **PROGRAMADO**.

Flujo principal:

1. El enfermero accede al turno asignado.
2. Registra la cantidad encontrada de cada insumo.
3. El sistema registra las cantidades de ingreso y determina los faltantes correspondientes.
4. El enfermero registra las observaciones de ingreso cuando corresponda.

5. Confirma el control de ingreso.
6. El sistema cambia el estado del turno a **EN_CURSO**.

Flujos alternativos y de excepción:

- Si el turno no se encuentra en estado **PROGRAMADO**, el sistema rechaza el registro del ingreso.
- Si se produce un error durante el registro, el sistema informa que el control de ingreso no pudo completarse.

Postcondición: El control inicial queda registrado y el turno pasa al estado **EN_CURSO**.

HU-03 – Control de Egreso

Historia: Como enfermero, quiero registrar las cantidades restantes al finalizar el turno y las reposiciones realizadas durante la guardia, para conservar el registro del consumo de insumos.

Actor: Enfermero.

Precondiciones:

- El enfermero debe estar autenticado.
- El turno debe estar asignado al enfermero.
- El turno debe encontrarse en estado **EN_CURSO**.
- Debe existir un control de ingreso registrado.

Flujo principal:

1. El enfermero registra la cantidad restante de cada insumo.
2. Registra las cantidades repuestas durante el turno cuando corresponda.
3. El sistema determina la cantidad utilizada a partir del ingreso, las reposiciones y el egreso.
4. El enfermero registra las observaciones de egreso cuando corresponda.
5. Confirma el control de egreso.
6. El sistema cambia el estado del turno a **FINALIZADO**.

Flujos alternativos y de excepción:

- Si el turno no se encuentra en estado **EN_CURSO**, el sistema rechaza el registro del egreso.

- Si se produce un error durante el registro, el turno no se finaliza.

Postcondición: El control de egreso queda registrado junto con las cantidades utilizadas y el turno pasa al estado **FINALIZADO**.

HU-04 – Consultar Historial del Turno

Historia: Como usuario autorizado, quiero consultar la información registrada durante un turno, para revisar su estado operativo, los faltantes detectados, las reposiciones, los consumos y las observaciones.

Actores: Administrador y Enfermero.

Precondiciones:

- El usuario debe estar autenticado.
- El turno debe existir en el sistema.
- El usuario debe tener acceso al turno correspondiente.

Flujo principal:

1. El usuario accede al turno.
2. El sistema muestra la información general del turno.
3. Se visualizan los faltantes detectados al ingreso cuando existen.
4. Se consultan los insumos utilizados y las reposiciones registradas.
5. Se consultan las observaciones y el estado del turno.

Flujos alternativos y de excepción:

- Si no existen faltantes al ingreso, el sistema informa que no se registraron faltantes.
- Si un turno finalizado no presenta novedades de insumos, el sistema informa que no existen novedades registradas.

Postcondición: El usuario obtiene la información histórica disponible del turno sin modificar sus datos operativos.

---

3. Marco teórico y tecnologías seleccionadas

3.1 Marco teórico

El desarrollo de Control107 se apoya en conceptos de arquitectura de software, desarrollo web, persistencia relacional y seguridad que permiten organizar las responsabilidades del sistema y mantener la trazabilidad de la información operativa.

Arquitectura cliente-servidor y API REST

En Control107 se separó la interfaz que utiliza el usuario de la lógica encargada de procesar y almacenar la información. El frontend se comunica con el backend a través de una API REST, enviando solicitudes HTTP a los distintos endpoints y recibiendo los datos necesarios en formato JSON. De esta manera, cada parte de la aplicación mantiene una responsabilidad diferenciada.

Arquitectura en capas y separación de responsabilidades

El backend de Control107 se organizó en capas para mantener separadas las distintas responsabilidades del sistema. Los controladores reciben las solicitudes realizadas a la API, los servicios concentran la lógica y las validaciones del dominio, y los repositorios se encargan del acceso a los datos. Por otro lado, las entidades representan la información persistida y los objetos de transferencia de datos (DTOs) permiten estructurar la información que se intercambia y decidir qué datos se comparten entre las distintas partes del sistema y hacia el exterior.

Esta organización permite mantener cada responsabilidad en un lugar definido y facilita comprender y modificar el código sin mezclar la lógica del sistema con el acceso a datos o la comunicación con el frontend.

Aplicación de página única y modularidad del frontend

En una aplicación de página única (SPA), la interacción y los cambios de vista se realizan principalmente desde el frontend, evitando depender de una carga completa de la página para cada navegación. Este enfoque se tomó como base para desarrollar la interfaz web de Control107.

A partir de esta decisión, el frontend se organizó de forma modular, separando páginas, componentes reutilizables, servicios, hooks, interfaces, stores y utilidades. Cada parte concentra una responsabilidad diferente: las páginas organizan las vistas principales, los

componentes permiten reutilizar elementos de la interfaz y los servicios concentran la comunicación con la API, mientras que el resto de los módulos permite separar el estado, los tipos de datos y la lógica reutilizable.

Esta organización permitió mantener separada la presentación de la comunicación con el backend y evitar concentrar toda la lógica de la interfaz en los mismos componentes.

Modelo de datos relacional e integridad

Un modelo de datos relacional es adecuado para este sistema porque la información que maneja se encuentra vinculada entre sí. Los usuarios, ambulancias, categorías, insumos, turnos y controles de insumos no funcionan como registros aislados, sino que mantienen relaciones necesarias para representar cada guardia y su operación.

A partir de estas relaciones, el sistema puede conservar qué ambulancia y qué personal participaron en un turno, qué insumos fueron controlados y qué información se registró durante el ingreso y el egreso. Mantener estas asociaciones de forma consistente permite preservar el historial y la trazabilidad de la operación.

Autenticación y autorización

En Control107 no todos los usuarios deben acceder a las mismas funcionalidades, por lo que fue necesario identificar quién inicia sesión y qué operaciones puede realizar. Para resolverlo, el sistema utiliza JWT para mantener la identidad del usuario autenticado y conocer su rol durante la sesión.

A partir de esta información, se diferencian los permisos de los roles ADMINISTRADOR y ENFERMERO. El administrador accede a las funciones de gestión y consulta general del sistema, mientras que el enfermero utiliza las funcionalidades correspondientes al control operativo de sus turnos. La autorización se aplica en el backend, evitando que el acceso a una operación dependa únicamente de lo que muestra la interfaz.

De esta manera, la autenticación permite reconocer al usuario que utiliza el sistema y la autorización determina qué acciones puede realizar de acuerdo con su rol.

3.2 Tecnologías seleccionadas y justificación

Las tecnologías utilizadas en Control107 fueron seleccionadas considerando las necesidades del sistema, la experiencia de desarrollo, la integración entre sus componentes y las características de la infraestructura disponible. También se tuvo en

cuenta mantener una arquitectura que pudiera desplegarse en un VPS con recursos limitados.

Java 17

- **Propósito:** lenguaje utilizado para desarrollar el backend de Control107.
- **Justificación:** se optó por Java 17 como una versión estable y suficiente para las necesidades del proyecto. Aunque se evaluó utilizar una versión más reciente, se priorizó una configuración adecuada para el despliegue previsto en un VPS con recursos limitados.
- **Alternativas consideradas:** versiones más recientes de Java.
- **Licencia:** GPL v2 con Classpath Exception (OpenJDK).

Spring Boot 4.1.1

- **Propósito:** framework principal del backend, utilizado junto con Spring Data JPA para la persistencia y Spring Security para la autenticación y autorización.
- **Justificación:** se seleccionó como base del backend porque permite integrar la API REST, la persistencia y la seguridad dentro del mismo ecosistema Java y mantener la organización en capas definida para el proyecto.
- **Alternativas consideradas:** no se evaluó otro framework principal para el backend.
- **Licencia:** Apache License 2.0.

MySQL 8.0

- **Propósito:** base de datos relacional utilizada para persistir la información de Control107.
- **Justificación:** se buscó una solución relacional que cubriera las necesidades de integridad y relaciones del modelo de datos y que pudiera configurarse teniendo en cuenta los recursos disponibles en el VPS.
- **Alternativas consideradas:** PostgreSQL. Se optó por MySQL considerando las restricciones de infraestructura definidas para el despliegue.
- **Licencia:** GPL v2.

React 19.3.1 + TypeScript 6.0.3 + Tailwind CSS 4.3.3

- **Propósito:** stack principal utilizado para desarrollar la interfaz web de Control107.
- **Justificación:** React permite organizar la interfaz mediante componentes reutilizables, TypeScript aporta tipado al código y a los datos, y Tailwind CSS

permite desarrollar los estilos mediante clases de utilidad manteniendo un diseño propio.

- **Alternativas consideradas:** para los estilos se consideró Bootstrap, pero se prefirió Tailwind CSS para mantener mayor control sobre la identidad visual y evitar depender de componentes prediseñados.
- **Licencia:**
    - **React y Tailwind:** MIT.
    - TypeScript: **Apache License 2.0**

Vite 8.3.4

- **Propósito:** herramienta utilizada para ejecutar el entorno de desarrollo y generar la compilación del frontend.
- **Justificación:** se seleccionó por su integración con React y por proporcionar el flujo de desarrollo y construcción requerido por el frontend.
- **Alternativas consideradas:** no se realizó una evaluación formal de otra herramienta para esta versión del proyecto.
- **Licencia:** MIT.

Docker / Docker Compose

- **Propósito:** contenerización y coordinación de los servicios necesarios para ejecutar el backend y su infraestructura.
- **Justificación:** permite definir un entorno de ejecución reproducible y coordinar los servicios que componen el despliegue, incluyendo la API y MySQL.
- **Alternativas consideradas:** no se evaluó otra plataforma de contenerización. Ya que Docker es una tecnología consolidada y ampliamente utilizada en la industria.
- **Licencia:** depende de los componentes y distribución utilizados.

Caddy 2

- **Propósito:** servidor web utilizado como proxy inverso del backend.
- **Justificación:** se incorporó para exponer la API y redirigir las solicitudes hacia el servicio de Control107, manteniendo esta responsabilidad separada del backend.
- **Alternativas consideradas:** no se evaluó formalmente otro proxy para la versión implementada.
- **Licencia:** Apache License 2.0.

4. Descripción del proceso de desarrollo

El desarrollo de Control107 se realizó de forma incremental, comenzando por el relevamiento y diseño conjunto del dominio, continuando con la implementación del backend y avanzando posteriormente sobre el frontend y la integración del sistema. Durante este proceso fue necesario revisar decisiones iniciales y adaptar el alcance a las restricciones detectadas.

4.1 Metodología de desarrollo adoptada

Se utilizó un enfoque incremental y adaptativo. El equipo definió conjuntamente el funcionamiento general, las entidades, sus relaciones y la arquitectura. Posteriormente, las funcionalidades se implementaron y verificaron progresivamente.

El backend se priorizó para concentrar las reglas de negocio y definir los endpoints y contratos utilizados por el frontend. Mientras este trabajo avanzaba, se utilizó una API JSON simple para comenzar el desarrollo de la interfaz. Con los contratos principales definidos, el frontend comenzó a integrarse con la API real.

Git y GitHub se utilizaron para el control de versiones y la integración del trabajo realizado.

4.2 Planificación y cronograma

El desarrollo estuvo condicionado por las entregas académicas y por los cambios surgidos durante el análisis del dominio. El historial del repositorio permite resumir el proceso en las siguientes etapas:

Tabla 3. Cronograma de desarrollo del proyecto

Etapa	Período	Trabajo principal
Definición y diseño	Agosto - septiembre de 2026	Propuesta, documentación, arquitectura, entidades y modelo de datos.
Redefinición del dominio	Septiembre de 2026	Ajuste del alcance, actores, trazabilidad y reglas operativas.

Etapa	Período	Trabajo principal
Implementación del backend	Septiembre - octubre de 2026	Persistencia, servicios, controladores, DTO, validaciones, seguridad y pruebas.
Frontend operativo	Octubre de 2026	Flujo del enfermero, controles de ingreso y egreso, cancelación y oxígeno.
Frontend administrativo	Octubre de 2026	Autenticación, usuarios, ambulancias, categorías, insumos, turnos y dashboard.
Integración y estabilización	Finales de octubre de 2026	Adaptación de contratos, seguridad, manejo de errores y ajustes de interfaz.
Despliegue inicial	30/10/2026	Primera publicación del sistema integrado en el entorno de despliegue.

Desvíos y replanificaciones

Durante el desarrollo se produjeron dos desvíos principales. El primero llevó a redefinir el MVP, ya que el alcance inicial resultaba demasiado amplio para los tiempos, recursos e información disponibles. Esto permitió concentrar el sistema en la gestión de turnos y el control de insumos.

El segundo surgió durante la integración entre frontend y backend, cuando la evolución de los DTO, las reglas de seguridad y algunos aspectos del dominio requirió adaptar funcionalidades ya desarrolladas. Ambos desvíos se resolvieron mediante ajustes progresivos, manteniendo el alcance final definido para el MVP.

4.3 Narrativa del proceso de desarrollo

4.3.1 Etapa de relevamiento y análisis

El análisis inicial permitió identificar que la propuesta original excedía las posibilidades del MVP y contemplaba procesos para los cuales no existían procedimientos, información o infraestructura suficiente. El equipo redefinió el alcance y concentró el sistema en la gestión de turnos, ambulancias, insumos y la trazabilidad de los controles realizados durante las guardias.

4.3.2 Etapa de diseño

El diseño se realizó en conjunto, definiendo entidades, relaciones, reglas de negocio y arquitectura. Durante esta etapa se simplificó el modelo de personal, se integró el oxígeno dentro del catálogo de insumos críticos y se descartó la comparación automática entre turnos debido al proceso externo de reposición existente.

También quedaron fuera del MVP las intervenciones clínicas detalladas y el control mecánico completo de los vehículos.

4.3.3 Etapas de implementación

La implementación comenzó por el backend, donde se concentraron las reglas del dominio y se definieron los contratos utilizados por el frontend.

Posteriormente se realizó la integración con el backend real. Durante este proceso fue necesario adaptar componentes del frontend debido a cambios en DTO, seguridad y reglas del dominio. Las últimas iteraciones completaron los flujos operativo y administrativo, junto con correcciones de interfaz y manejo de errores.

4.3.4 Etapa de pruebas y validación

Las pruebas acompañaron el desarrollo del sistema. En el backend se realizaron pruebas sobre servicios, controladores, validaciones y reglas del dominio. La integración permitió además verificar los principales flujos entre frontend y backend. La estrategia y los resultados se detallan en la sección de pruebas del informe.

4.3.5 Etapa de despliegue

Una vez integrado el sistema, el frontend se publicó en Vercel y el backend en un VPS expuesto mediante HTTPS. El despliegue permitió verificar el funcionamiento de ambos componentes en el entorno publicado.

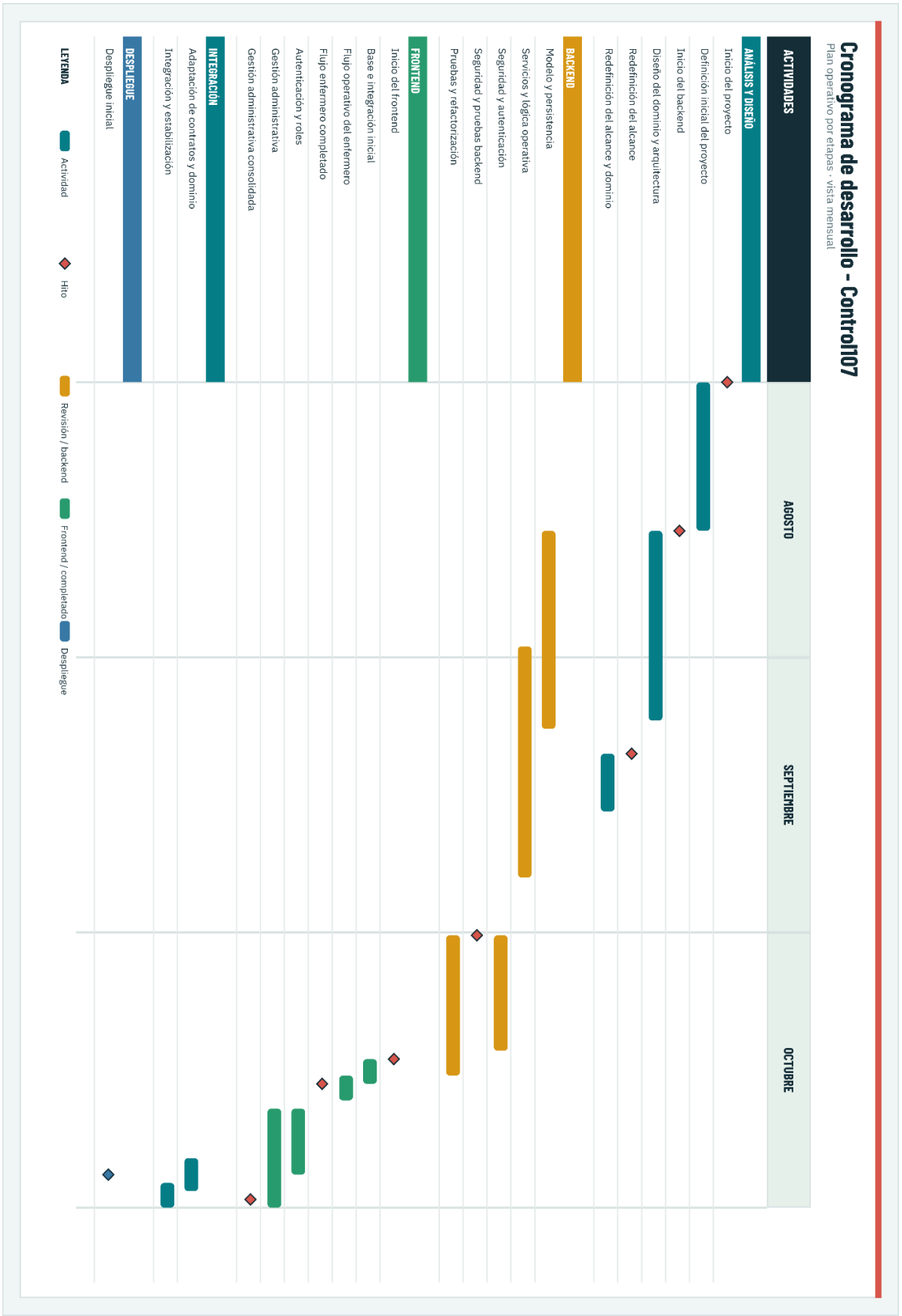
4.4 Control de versiones

El desarrollo se gestionó mediante Git y un repositorio compartido en GitHub. El diseño inicial del sistema se realizó en conjunto y, durante la implementación, el trabajo se distribuyó principalmente entre backend y frontend, utilizando ramas diferenciadas que permitieron avanzar sobre ambos componentes e integrarlos progresivamente.

El historial refleja una implementación incremental mediante commits asociados a funcionalidades concretas, correcciones, pruebas y refactorizaciones. Durante el desarrollo del frontend se incorporaron periódicamente los cambios realizados en el backend para trabajar sobre los contratos y endpoints actualizados. Esta forma de trabajo permitió detectar durante la integración incompatibilidades en DTO, seguridad y reglas del dominio, que posteriormente dieron lugar a nuevas correcciones en la interfaz.

También se utilizaron merges y pull requests para incorporar cambios entre ramas y consolidar distintas etapas del proyecto. No se adoptó una convención formal de commits ni una estrategia de branching estricta; las ramas se utilizaron de forma práctica para separar áreas de trabajo y mantener estable el código mientras avanzaba la implementación.

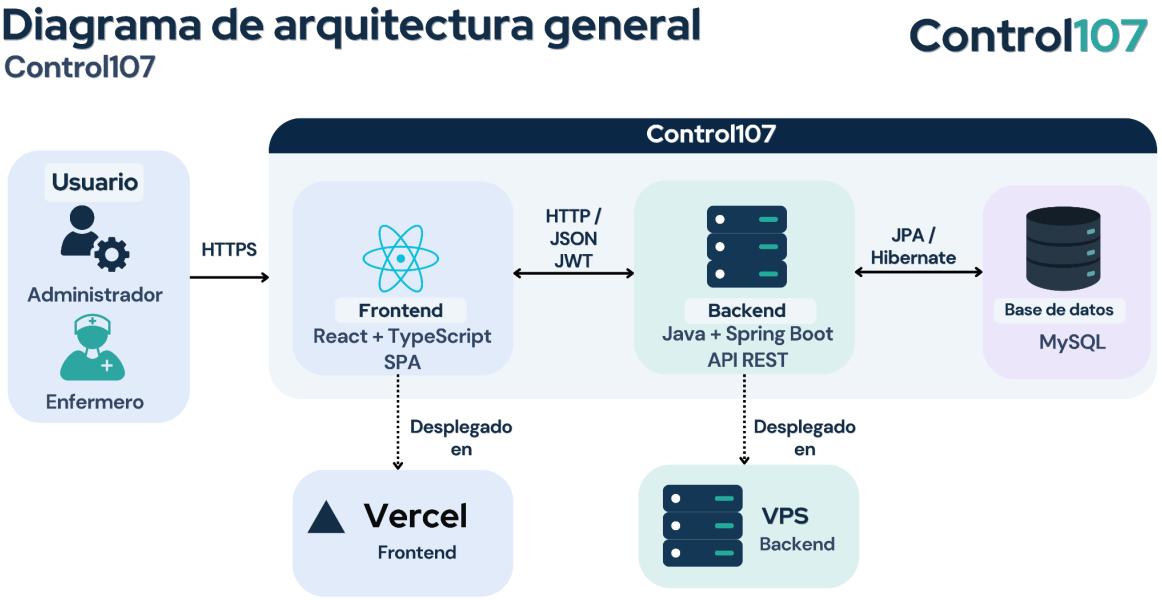
Figura 1. Diagrama de Gantt



5. Arquitectura y decisiones de diseño

5.1 Diagrama de arquitectura general

Figura 2. Arquitectura general de Control107.



Control107 utiliza una arquitectura cliente-servidor compuesta por una aplicación web de página única que consume una API REST. El backend concentra las reglas del dominio y gestiona el acceso a la persistencia mediante JPA/Hibernate sobre una base de datos relacional. El acceso a las funcionalidades protegidas utiliza autenticación mediante JWT. En el entorno desplegado, el frontend se publica en Vercel y el backend se ejecuta en un VPS expuesto mediante HTTPS.

5.2 Descripción de la arquitectura

El sistema adopta una arquitectura cliente-servidor con una separación clara entre frontend, backend y persistencia. El frontend constituye una SPA desarrollada con React y TypeScript que se comunica con el backend mediante HTTP/JSON. Esta separación permite mantener la interfaz desacoplada de las reglas del dominio.

El backend se organiza mediante una arquitectura en capas. Los controladores reciben las solicitudes HTTP y exponen los endpoints REST, los servicios concentran la lógica y las validaciones del dominio, y los repositorios gestionan el acceso a los datos mediante Spring Data JPA. Las entidades representan el modelo persistente y los DTO permiten

organizar la información y definir qué datos se comparten dentro del sistema y cuáles se exponen hacia el exterior.

Esta organización fue seleccionada para mantener separadas las responsabilidades del sistema y facilitar la modificación o extensión de sus componentes dentro del alcance del MVP.

5.3 Decisiones de diseño relevantes

Para documentar las principales decisiones se utiliza el formato Architecture Decision Record (ADR). En el informe se presenta una versión resumida de tres decisiones centrales. Los registros completos se mantienen como documentación técnica dentro del repositorio.

ADR-01 – Arquitectura monolítica en capas

Contexto: El proyecto requería una arquitectura que permitiera separar las reglas del dominio, el acceso a los datos y la exposición de la API sin incorporar una infraestructura innecesariamente compleja para el alcance del MVP.

Opciones consideradas: Arquitectura monolítica en capas y arquitectura basada en microservicios.

Decisión adoptada: Se implementó un backend monolítico organizado en capas Controller, Service, Repository y modelo de dominio.

Consecuencias: La solución mantiene responsabilidades diferenciadas dentro de una única aplicación desplegable y reduce la complejidad de infraestructura. Como contrapartida, los módulos del backend comparten el mismo proceso de aplicación.

ADR-02 – Persistencia mediante base de datos relacional

Contexto: El sistema necesita mantener relaciones consistentes entre usuarios, turnos, ambulancias, insumos y controles operativos.

Opciones consideradas: Base de datos relacional y base de datos documental.

Decisión adoptada: Se seleccionó un modelo relacional utilizando MySQL para el entorno de producción.

Consecuencias: El modelo permite representar explícitamente las relaciones entre las entidades y aplicar restricciones para conservar la integridad de los datos. La estructura

requiere mantener un esquema definido y controlar sus modificaciones a medida que evoluciona el dominio.

ADR-03 – Simplificación del modelo operativo del dominio

Contexto: La propuesta inicial contemplaba un modelo de personal y una trazabilidad entre turnos más amplios que los procesos que podían representarse de manera verificable dentro del MVP.

Opciones consideradas: Mantener el modelo inicial con múltiples participantes y comparación entre turnos, o concentrar el sistema en los actores que interactúan con la aplicación y en la trazabilidad interna de cada turno.

Decisión adoptada: Se definieron Administrador y Enfermero como roles del sistema. Médico y chofer se registran como información asociada al turno. La trazabilidad de insumos se concentra en los controles de ingreso, reposición y egreso realizados dentro de cada turno.

Consecuencias: El modelo reduce la complejidad del dominio y permite representar el proceso para el cual existe información verificable. Las funcionalidades relacionadas con equipos variables y comparación automática entre turnos quedan fuera del alcance del MVP.

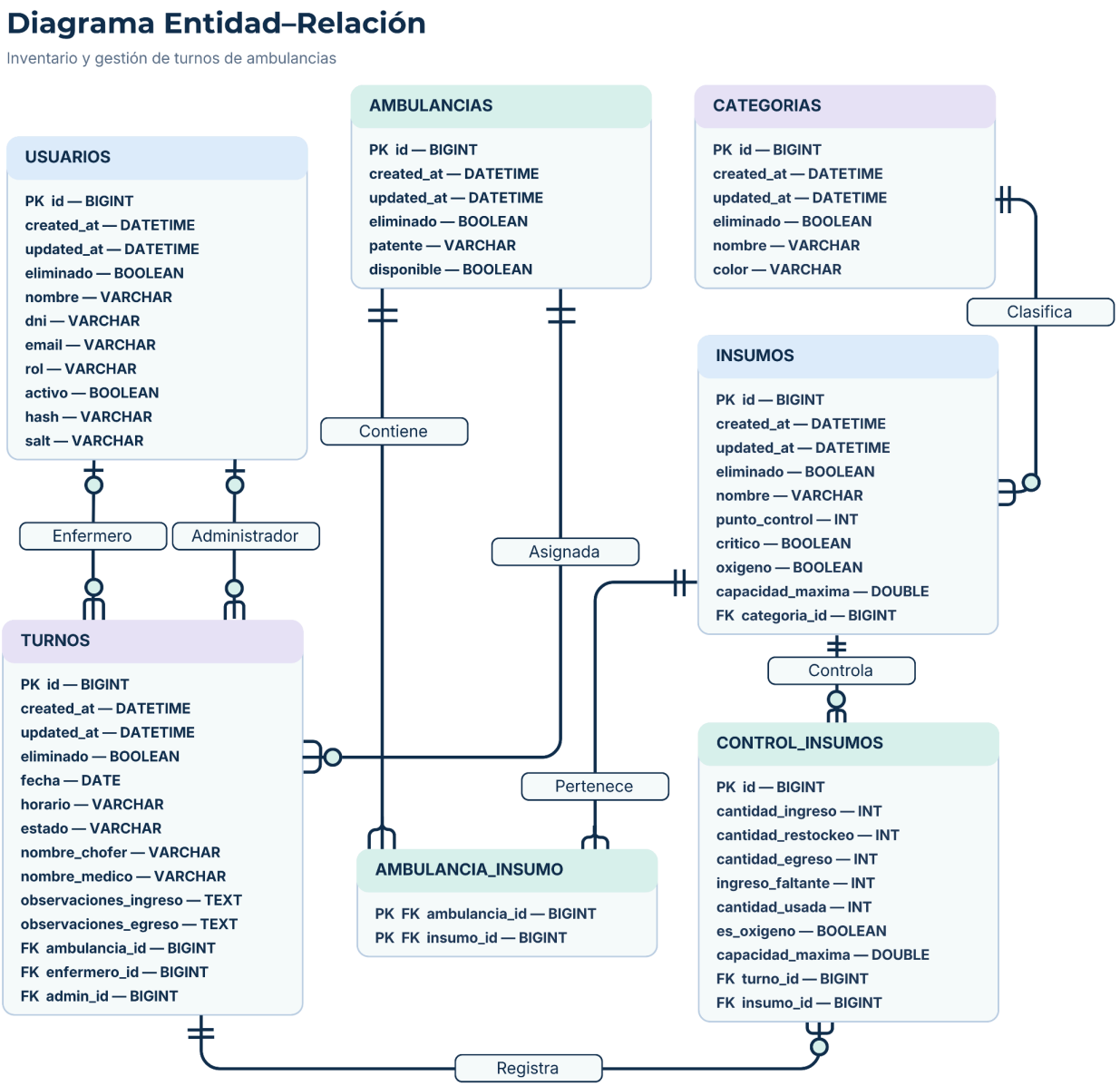
5.4 Modelo de datos

El sistema utiliza un modelo relacional compuesto por las entidades principales **Usuario**, **Ambulancia**, **Categoría**, **Insumo**, **Turno** y **Control de Insumo**, junto con la relación que permite establecer los insumos asignados a cada ambulancia.

Los usuarios con roles Administrador y Enfermero participan en los turnos según sus responsabilidades. Cada turno se encuentra asociado con una ambulancia y registra sus controles de insumos. A su vez, cada insumo pertenece a una categoría y puede formar parte de la dotación asignada a una o más ambulancias.

Control de Insumo representa el registro operativo de un insumo dentro de un turno y conserva las cantidades de ingreso, reposición y egreso, junto con los valores utilizados para representar faltantes y consumo. El modelo contempla además el tratamiento del oxígeno dentro del catálogo de insumos.

Figura 3. Diagrama Entidad-Relación de Control107.

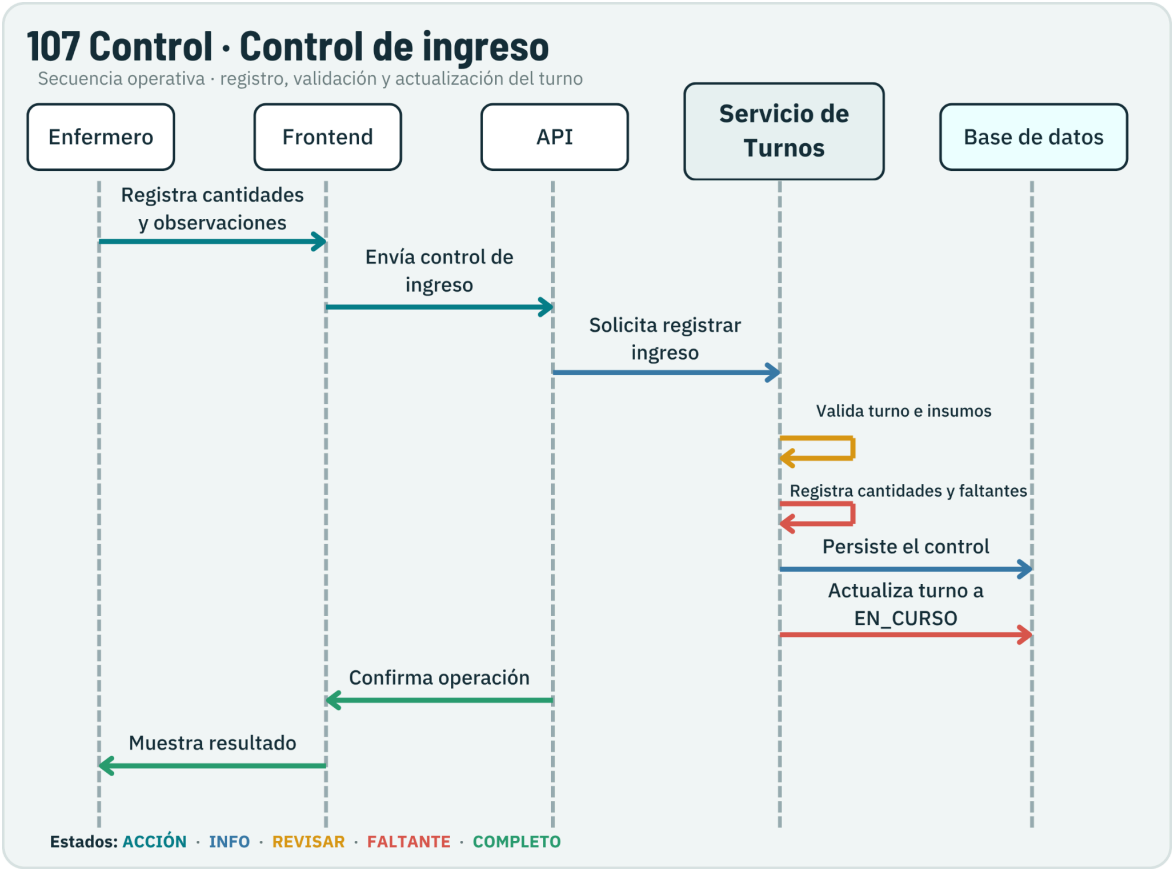


5.5 Diagramas de secuencia o flujo

Se seleccionaron los procesos de control de ingreso y control de egreso por representar los principales flujos operativos realizados por el enfermero.

5.5.1 Control de ingreso

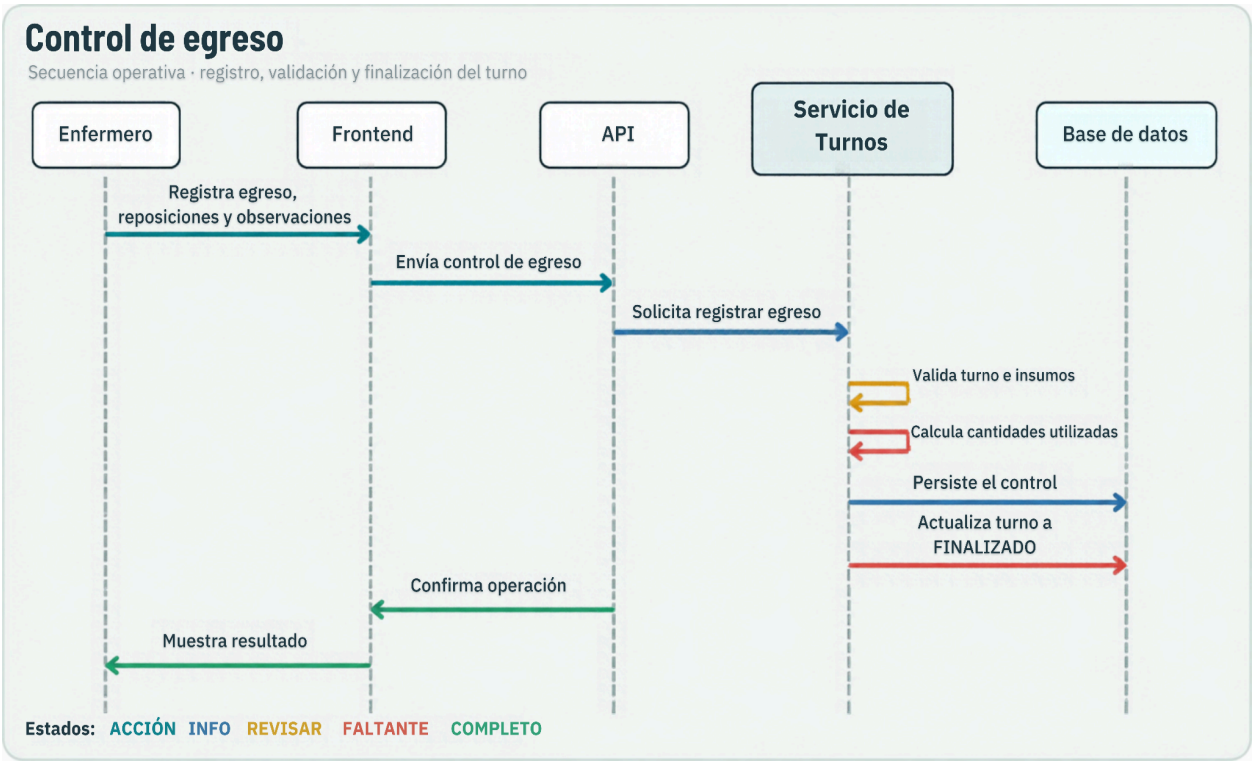
Figura 4. Secuencia del control de ingreso.



El control de ingreso registra las cantidades encontradas por el enfermero al comenzar la guardia. El backend valida que la operación pueda realizarse sobre el turno correspondiente, registra la información de los insumos y actualiza el estado del turno a **EN_CURSO**.

5.5.2 Control de egreso

Figura 5. Secuencia del control de egreso.



El control de egreso registra las cantidades restantes y las reposiciones realizadas durante la guardia. A partir de estos datos y del ingreso previamente registrado, el dominio determina las cantidades utilizadas. Una vez persistido el control, el turno pasa al estado **FINALIZADO**.

5.6 Diseño de la interfaz de usuario

La interfaz se diseñó diferenciando los flujos correspondientes a Administrador y Enfermero. El área administrativa concentra la gestión de usuarios, ambulancias, categorías, insumos, turnos y estadísticas, mientras que la interfaz del enfermero prioriza las operaciones necesarias durante el desarrollo de una guardia.

La identificación visual utiliza colores asociados a las categorías y señales específicas para los insumos críticos. El oxígeno recibe además un tratamiento visual diferenciado debido a su importancia operativa

La interfaz operativa del enfermero fue diseñada con comportamiento responsive para facilitar su utilización en dispositivos de distintos tamaños. Para las tareas administrativas se prioriza su utilización desde equipos de escritorio o notebooks debido a la cantidad de información y controles disponibles.

Figura 6. Interfaz operativa del Enfermero.

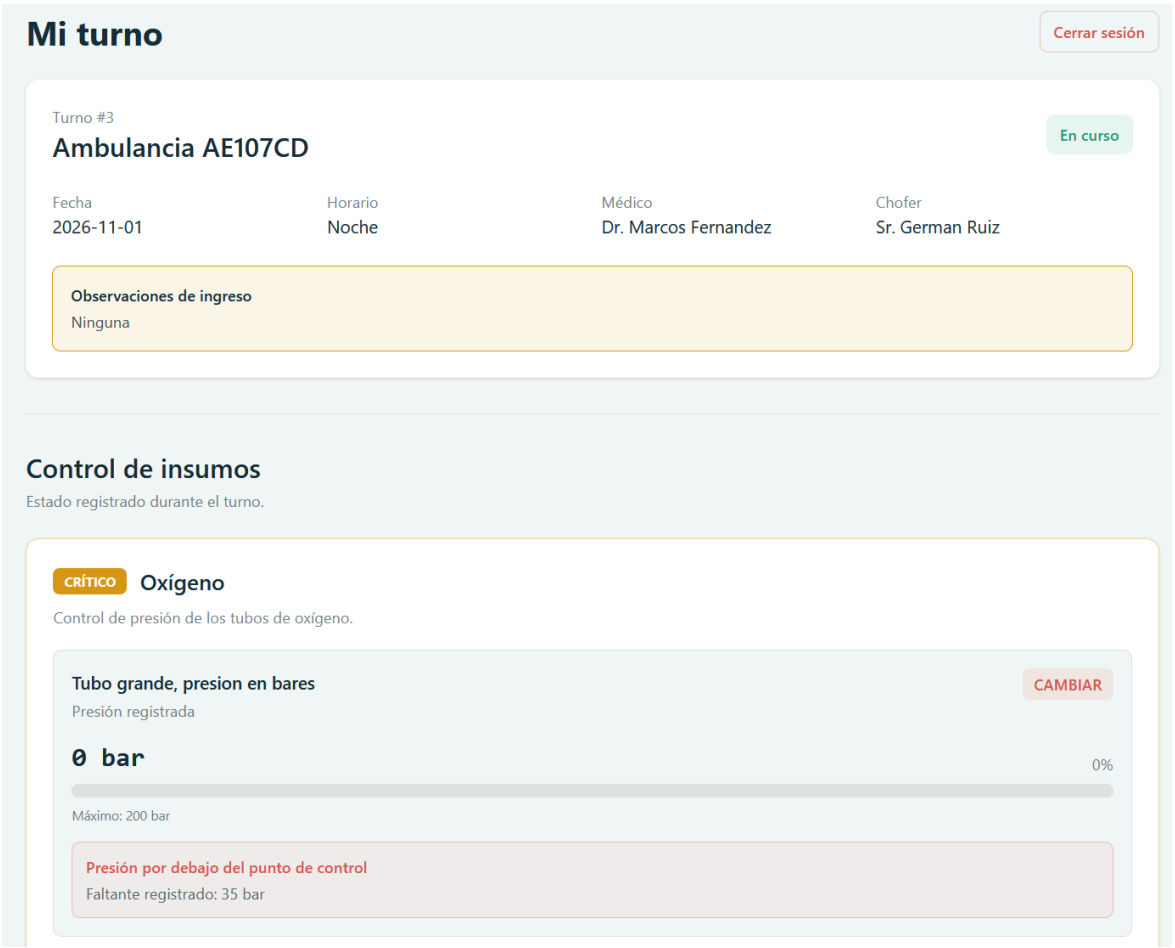
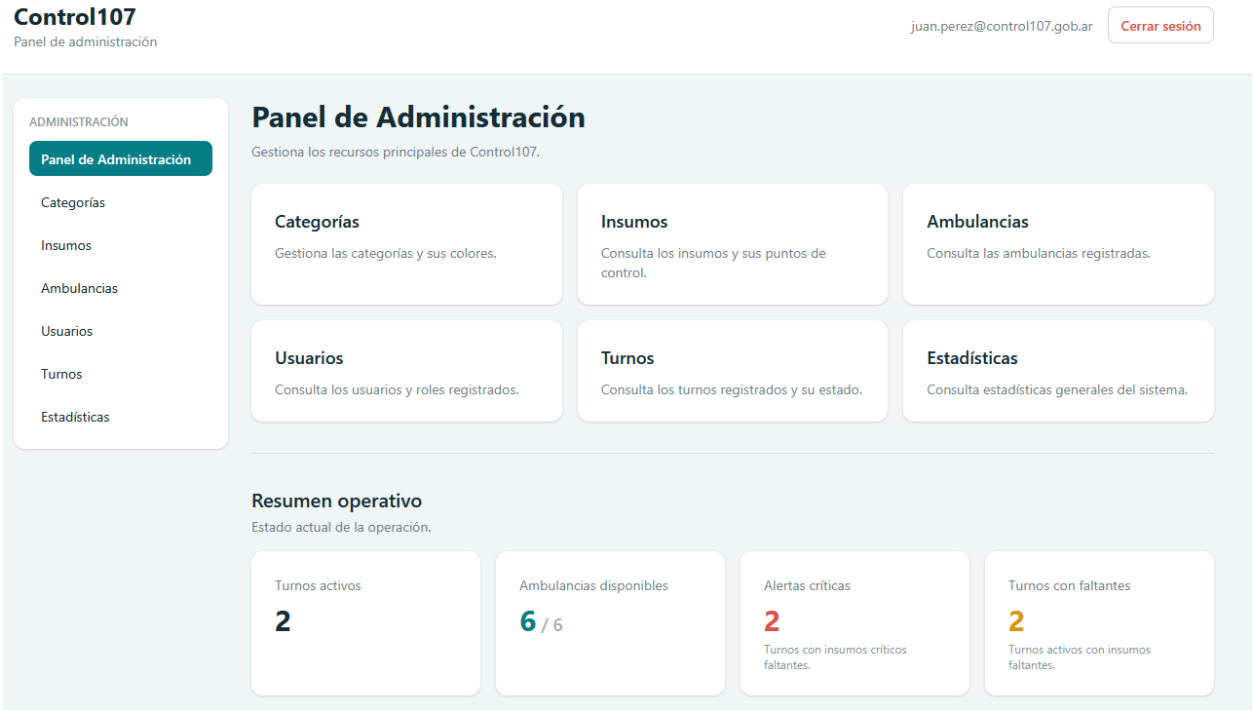


Figura 7. Interfaz de administración.



6. Implementación del sistema

La implementación de Control107 se divide en un backend encargado de la lógica de negocio, seguridad y persistencia, y un frontend responsable de la interacción con los usuarios y el consumo de la API REST.

6.1 Estructura del proyecto

El proyecto mantiene separados el backend, frontend, base de datos y documentación:

```
None
UTN-TUPaD-TFI/
├── backend/                        # API REST y lógica de negocio
│   └── control107/
│       ├── src/main/               # Modulos de la aplicacion
│       └── src/test/               # Pruebas automatizadas del backend
│
├── frontend/                       # Aplicación web - React y TypeScript
│   └── control107/
│       └── src/
│           ├── components/         # Componentes visuales reutilizables
│           ├── hooks/              # Lógica para consultas y mutaciones
│           ├── interfaces/         # Tipos e interfaces de TypeScript
│           ├── pages/              # Vistas principales de la aplicación
│           ├── services/           # Comunicación con la API
│           ├── stores/             # Estado global y autenticación
│           └── utils/              # Funciones auxiliares
│
├── database/                       # Scripts de inicialización de la BD
└── docs/                           # Documentación técnica del proyecto
```

El **backend** organiza el código en controladores, servicios, repositorios, entidades, DTO y configuración. El **frontend** separa componentes, páginas, servicios de comunicación con la API, hooks, interfaces y estado global. Esta estructura mantiene separadas las responsabilidades de presentación, lógica de negocio y acceso a datos.

6.2 Módulos o componentes principales

6.2.1 Autenticación y seguridad

El backend autentica a los usuarios mediante JWT y restringe las operaciones según el rol. Algunas acciones agregan una validación adicional para garantizar que el enfermero esté asignado al turno correspondiente.

```java
@PostMapping("/ingreso")
@PreAuthorize(
    "@securityUtil.esEnfermeroAsignado(#request.turnoId, authentication)"
)
public ResponseEntity<Void> registrarIngreso(
        @RequestBody IngresoEgresoDTO request) {
    turnoService.registrarIngreso(request);
    return ResponseEntity.ok().build();
}
```

En el frontend, el estado de autenticación administra el token y los datos del usuario. Axios agrega automáticamente el token a las solicitudes protegidas y React Router restringe las rutas según el rol.

6.2.2 Gestión de turnos

El módulo de turnos administra su creación, modificación, consulta, cancelación y las operaciones de ingreso y egreso. Los estados **PROGRAMADO**, **EN_CURSO**, **FINALIZADO** y **CANCELADO** determinan qué operaciones pueden realizarse.

El frontend adapta la vista operativa al estado del turno: un turno programado permite realizar el control de ingreso o cancelarlo, mientras que uno en curso habilita el control de insumos y el registro de egreso.

6.2.3 Control de insumos

El control de insumos registra las cantidades encontradas al inicio y al final de la guardia. El backend calcula automáticamente los faltantes y el consumo:

```java
public void informarIngreso(int cantidadIngreso) {
    this.cantidadIngreso = cantidadIngreso;
    this.ingresoFaltante = Math.max(
        0, this.insumo.getPuntoControl() - cantidadIngreso
    );
}

public void informarEgreso(int egreso, int reposicion) {
    this.cantidadEgreso = egreso;
    this.cantidadRestockeo = reposicion;
    this.cantidadUsada = Math.max(
        0, this.cantidadIngreso + reposicion - egreso
    );
}
```

De esta forma, los valores derivados se calculan a partir de los datos registrados y no deben ser ingresados manualmente.

6.2.4 Tratamiento del oxígeno

El oxígeno se identifica de forma independiente de los demás insumos para permitir una representación específica en la interfaz:

```typescript
const controlesOxigeno =
    controles.filter((control) => control.esOxigeno);

const controlesCriticos = controles.filter(
    (control) =>
        control.esCritico && !control.esOxigeno,
);

const controlesNormales = controles.filter(
    (control) => !control.esCritico,
);
```

Además, durante el ingreso se controla que la presión registrada no supere la capacidad máxima:

```typescript
if (
    esOxigeno &&
    value > control.capacidadMaxima
) {
    return `La presión no puede superar
        ${control.capacidadMaxima} bar`;
}
```

Esta información también se utiliza para representar gráficamente el nivel de presión y diferenciar el oxígeno de los demás insumos controlados.

6.2.5 Administración y resumen operativo

El panel administrativo reúne la gestión de **usuarios, categorías, insumos, ambulancias y turnos**, utilizando servicios y componentes con una estructura común para las operaciones CRUD.

También dispone de un resumen operativo que reúne información sobre turnos activos, disponibilidad de ambulancias, alertas e inconsistencias, facilitando una visión general del estado del sistema.

6.3 Integración con sistemas externos

Control107 no depende de APIs externas de terceros para su funcionamiento principal. Las integraciones técnicas se concentran en la comunicación entre los componentes de la propia aplicación y los servicios utilizados para su ejecución.

API REST: el frontend se comunica con el backend mediante HTTP y JSON utilizando Axios. Las solicitudes protegidas incorporan un token JWT con el esquema **Bearer**. Los errores de la API son tratados de forma centralizada en el backend y gestionados por el frontend.

Base de datos: el backend utiliza MySQL para la persistencia y H2 en memoria para las pruebas automatizadas. El frontend no accede directamente a la base de datos, sino que todas las operaciones pasan por la API.

OpenAPI/Swagger: la API utiliza Springdoc OpenAPI para documentar y probar los endpoints, incluyendo el esquema de autenticación mediante JWT.

Docker: el proyecto utiliza contenedores para separar la ejecución de la API, la base de datos MySQL y Caddy. Este último funciona como proxy inverso, recibiendo las solicitudes dirigidas a la API y redirigiéndolas hacia el servicio backend.

---

7. Manual de usuario

Esta sección presenta una guía resumida para el uso de Control107. El procedimiento detallado de cada funcionalidad, junto con capturas adicionales, se encuentra disponible en el **Manual de Usuario completo**, entregado como documentación complementaria. Es posible verlo en el siguiente link.

https://drive.google.com/file/d/1luQCK37BpFZXU0GnahXSscmGrfHvU6Cx/view?usp=sharing

7.1 Introducción al manual

Control107 es una aplicación web destinada a la gestión de turnos, ambulancias y controles de insumos del servicio de emergencias. El sistema dispone de dos perfiles de usuario: **administrador** y **enfermero**, cada uno con funciones específicas.

El administrador gestiona usuarios, ambulancias, categorías, insumos y turnos, además de consultar información operativa e histórica. El enfermero accede a sus turnos asignados y realiza los controles correspondientes al inicio y al final de la guardia.

Figura 8. Formulario de Login



7.2 Acceso al sistema

Acceda a Control107 desde:

https://control107.vercel.app

Para iniciar sesión:

1. Ingrese su correo electrónico.
2. Ingrese su contraseña.
3. Haga clic en **Iniciar sesión**.
4. El sistema mostrará las opciones correspondientes a su rol.

Las cuentas de usuario son gestionadas por un administrador. No se dispone de registro público de usuarios.

7.3 Descripción de funcionalidades

Administrador

El perfil administrador permite gestionar la configuración y operación general del sistema.

- **Usuarios:** cree, modifique o desactive los usuarios del sistema.
- **Ambulancias:** registre ambulancias y seleccione los insumos asociados.
- **Categorías e insumos:** administre los elementos utilizados para realizar los controles.
- **Turnos:** cree y consulte turnos, asignando la ambulancia y el personal correspondiente.
- **Estadísticas:** consulte información resumida sobre la operación registrada.

Figura 9: Vista del panel de Administracion



Enfermero

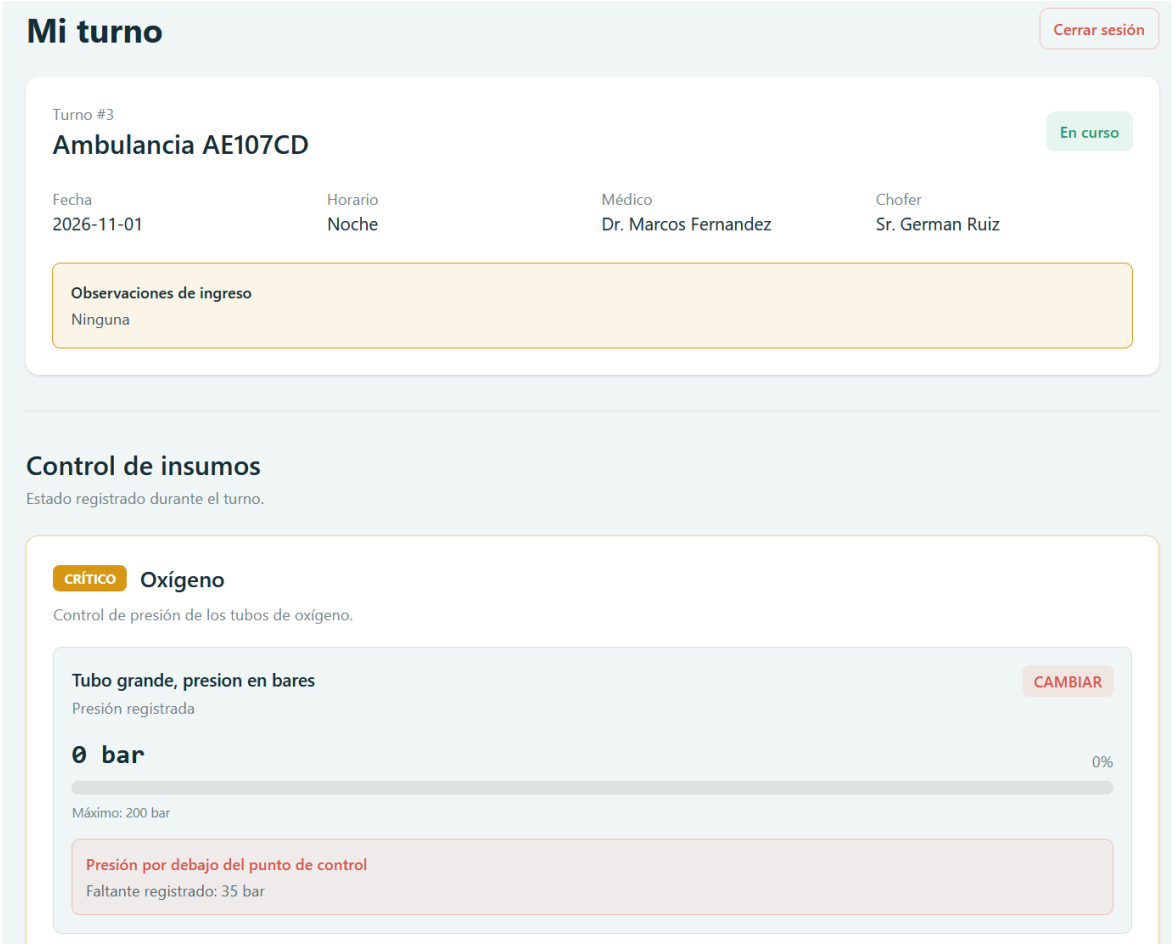
El perfil enfermero está orientado al control operativo de la guardia.

1. Ingrese al sistema y seleccione el turno asignado.
2. Registre las cantidades encontradas en el **control de ingreso**.
3. Inicie el turno y consulte el estado de los insumos.

4. Al finalizar la guardia, registre las cantidades de egreso y las reposiciones realizadas.
5. Confirme el egreso para finalizar el turno.

Los faltantes se calculan a partir de los puntos de control establecidos. Los insumos críticos se muestran de forma diferenciada y el oxígeno dispone de una representación específica de su presión.

Figura 10: Vista operativa del turno y control de insumos



Ante un mensaje de error, revise los datos ingresados y siga la indicación mostrada por el sistema. Si el problema impide continuar con la operación, comuníquese con el responsable de administración.

7.4 Preguntas frecuentes (FAQ)

¿Puedo crear mi propia cuenta?
No. Las cuentas son gestionadas por un usuario administrador.

¿Por qué no puedo acceder a una sección del sistema?
Las funcionalidades disponibles dependen del rol asignado al usuario.

¿Qué sucede si se detecta un faltante al iniciar el turno?
El sistema calcula el faltante comparando la cantidad registrada con el punto de control definido para el insumo.

¿Cómo se registra el oxígeno?
Ingrese la presión encontrada. El sistema utiliza **bar** como unidad y presenta el oxígeno de forma diferenciada.

¿Puedo modificar un turno que ya comenzó o finalizó?
Las operaciones disponibles dependen del estado actual del turno y de los permisos del usuario.

Para instrucciones detalladas sobre cada pantalla y operación, consulte el **Manual de Usuario completo** indicado al comienzo de esta sección.

---

8. Manual de implementación (despliegue)

Control107 separa su despliegue entre el frontend, publicado en **Vercel**, y el backend, ejecutado en un **VPS mediante Docker** junto con MySQL y Caddy. El despliegue del backend se automatiza mediante GitHub Actions ante cambios en la rama **main**.

8.1 Requisitos del entorno

Tabla 4. Requisitos del entorno

Categoría	Requisito
Sistema operativo	Ubuntu 24.04 (LTS) x64
Hardware	1 vCPU / 512 MB RAM / 10 GB Disk

Categoría	Requisito
Backend	Java 17, Docker y Docker Compose
Base de datos	MySQL 8.0
Proxy	Caddy 2
Puertos	80/443 públicos; 8080/3306 internos

El frontend se encuentra disponible en **https://control107.vercel.app**

8.2 Variables de entorno y configuración

Tabla 5. Variables del Entorno

Variable	Propósito
DB_ROOT_PASSWORD	Contraseña root de MySQL
DB_USERNAME	Usuario de la base de datos
DB_PASSWORD	Contraseña de la base de datos
JWT_SECRET_KEY	Clave utilizada para los tokens JWT
FRONTEND_HOST	Origen permitido para el frontend

El frontend utiliza **VITE_API_URL** para establecer la dirección de la API.

8.3 Procedimiento de instalación y despliegue

El despliegue del backend está automatizado mediante GitHub Actions. Ante cambios en **main**, el workflow configura Java 17, compila el proyecto:

```
Shell
mvn clean package
```

Una vez generado el artefacto, se copian al VPS el archivo JAR, docker-compose.yml, Dockerfile y Caddyfile. Posteriormente, el despliegue reconstruye los servicios mediante:

```shell
cd ~/control107
docker compose down
docker compose up -d --build
docker image prune -f
```

Docker Compose inicia los servicios correspondientes a **MySQL, la API y Caddy**. La aplicación se ejecuta utilizando el perfil de producción y Hibernate se encuentra configurado con **ddl-auto: update** para actualizar el esquema de la base de datos de acuerdo con las entidades del sistema.

El **DataLoader** se encuentra habilitado en producción y permite realizar la carga inicial de datos cuando la base no contiene usuarios. Si ya existen usuarios registrados, la carga se omite.

La información de MySQL se almacena en el volumen persistente **control107_db_data**. El procedimiento normal de despliegue detiene y reconstruye los contenedores sin eliminar este volumen, por lo que los datos se conservan entre despliegues.

8.4 Actualización del sistema

Las actualizaciones utilizan el mismo flujo automatizado al incorporar cambios a **main**, reconstruyendo los servicios sin eliminar los datos persistentes de MySQL.

El rollback no está automatizado, pero puede realizarse recuperando una versión anterior mediante **commits o tags de Git** y volviendo a desplegarla a través del mismo flujo.

8.5 Respaldo y recuperación

Los datos de MySQL se conservan mediante el volumen persistente **control107_db_data**. Actualmente no existe un procedimiento automatizado de backup y restauración, por lo que su incorporación queda planteada como mejora para producción.

8.6 Monitoreo y logs

No se utiliza actualmente una plataforma centralizada de monitorización. El estado y los registros de los servicios pueden consultarse mediante:

```shell
docker compose ps
docker compose logs -f api
docker compose logs -f db
docker compose logs -f caddy
```

Estos registros permiten revisar el funcionamiento de la API, la base de datos y el proxy inverso.

---

9. Pruebas y validación

9.1 Estrategia de pruebas

La estrategia de pruebas combinó **pruebas automatizadas en el backend** con verificaciones manuales durante el desarrollo e integración del frontend. Cada cambio significativo del frontend fue acompañado por una compilación y posteriormente se verificó su funcionamiento integrado con la API.

Tabla 6. Estrategia de pruebas

Tipo de prueba	Incluida
Pruebas unitarias	Sí
Pruebas de integración	Sí, parcialmente
Pruebas de sistema (E2E)	Manuales
Pruebas de usabilidad	No formalizadas
Pruebas de rendimiento	No
Pruebas de seguridad	Parciales

En el backend se utilizaron **JUnit, Mockito y MockMvc** para comprobar servicios, controladores, validaciones y respuestas HTTP. Para las pruebas que requieren persistencia se utilizó **H2** como base de datos de prueba.

Las verificaciones de seguridad se concentraron en los mecanismos de autenticación y autorización contemplados por las pruebas del backend. No se realizó un análisis de seguridad independiente.

9.2 Casos de prueba

Se seleccionaron los siguientes casos como representativos de las reglas de negocio y validaciones comprobadas:

Tabla 7. Casos de prueba representativos

ID	Caso	Resultado esperado
CP-01	Crear una ambulancia con patente duplicada	La operación es rechazada y no se persisten cambios
CP-02	Registrar el ingreso de un turno programado	El turno pasa a **EN_CURSO** y se calculan los faltantes
CP-03	Registrar un ingreso en un turno que ya está en curso	La operación es rechazada por estado inválido
CP-04	Modificar la ambulancia de un turno con controles de insumos asociados	La modificación es rechazada
CP-05	Solicitar un recurso inexistente mediante la API	Se devuelve una respuesta HTTP **404**

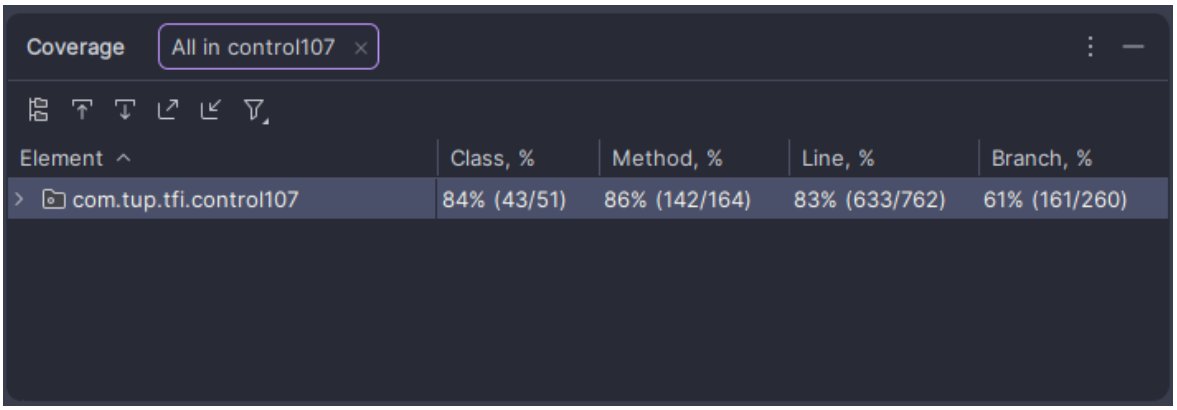
Las pruebas automatizadas también contemplan escenarios correctos y de error relacionados con usuarios, categorías, insumos, ambulancias y turnos. Durante la integración se verificó manualmente el funcionamiento completo entre frontend y backend.

9.3 Métricas de calidad

Las pruebas automatizadas del backend alcanzaron una cobertura del **83 % de líneas**, **86 % de métodos** y **84 % de clases**. La cobertura de ramas fue del **61 %**.

En el frontend, la calidad se controló mediante compilaciones frecuentes con TypeScript y Vite, análisis estático con Oxlint y verificaciones manuales de integración con el backend.

Figura 10. Cobertura de pruebas automatizadas del backend obtenida mediante IntelliJ IDEA



9.4 Validación con usuarios

No se realizaron sesiones formales de validación con usuarios finales. La validación funcional se realizó durante el desarrollo, comprobando progresivamente la integración entre frontend y backend y efectuando una verificación general del sistema una vez completadas las funcionalidades principales.

---

10. Conclusiones

El desarrollo de Control107 permitió llevar una necesidad operativa concreta a una solución funcional y, al mismo tiempo, identificar aspectos que podrían mejorarse tanto en el producto como en el proceso seguido para construirlo.

10.1 Resultados obtenidos

El objetivo general de desarrollar una aplicación web para digitalizar y centralizar la gestión de turnos, ambulancias y controles de insumos se considera **cumplido**, ya que se obtuvo un sistema funcional que permite registrar, conservar y consultar la información operativa asociada a las guardias.

Los objetivos específicos también se consideran cumplidos:

Tabla 8. Cumplimiento de los objetivos específicos

Objetivo específico	Resultado
Gestionar usuarios y sus responsabilidades	**Cumplido**
Administrar ambulancias y su dotación de insumos	**Cumplido**
Registrar los controles realizados durante los turnos	**Cumplido**
Detectar faltantes según los puntos de control	**Cumplido**
Conservar y consultar información histórica	**Cumplido**

El sistema también incorporó el tratamiento particular del oxígeno, la diferenciación de insumos críticos y la consulta de estadísticas básicas, manteniendo el alcance establecido para el producto mínimo viable.

10.2 Lecciones aprendidas

Uno de los principales aprendizajes fue la importancia de definir con claridad los contratos entre frontend y backend. Durante el desarrollo fue necesario ajustar estructuras de datos, endpoints y mecanismos de autenticación para mantener ambas partes integradas.

También resultó valioso avanzar de manera incremental, integrando las funcionalidades por etapas y verificando continuamente la compilación y el funcionamiento del sistema.

Si el proyecto comenzara nuevamente, se dedicaría más tiempo inicialmente a definir las reglas específicas del dominio y los contratos de la API antes de avanzar simultáneamente con frontend y backend, reduciendo modificaciones posteriores durante la integración.

10.3 Limitaciones del sistema actual

El sistema cumple con el alcance definido, aunque mantiene aspectos pendientes relacionados principalmente con su madurez como entorno productivo. Actualmente no se dispone de una estrategia automatizada de backup y recuperación, rollback automatizado ni monitorización centralizada.

Las pruebas E2E fueron realizadas manualmente y no se efectuaron pruebas formales de rendimiento o usabilidad con usuarios finales.

El backend dispone de pruebas automatizadas con una cobertura del **83 % de líneas, 86 % de métodos, 84 % de clases y 61 % de ramas**. La cobertura de ramas presenta el mayor margen de mejora mediante la incorporación de nuevos escenarios y caminos alternativos.

Las funcionalidades definidas fuera del alcance del producto mínimo viable no se consideran incumplimientos, sino posibles extensiones del sistema.

10.4 Trabajo futuro

Como evolución del proyecto se propone fortalecer principalmente los aspectos de operación y calidad mediante la automatización de backups y recuperación, rollback de despliegues y monitorización centralizada.

También se podrían incorporar pruebas E2E automatizadas, ampliar la cobertura de escenarios alternativos del backend, realizar pruebas de rendimiento y desarrollar sesiones de validación con usuarios finales.

Desde el punto de vista funcional, futuras versiones podrían incorporar funcionalidades actualmente fuera del alcance a medida que las necesidades operativas del servicio lo requieran.

Referencias bibliográficas

Caddy. (s. f.). *reverse_proxy (Caddyfile directive)*. Caddy Documentation. https://caddyserver.com/docs/caddyfile/directives/reverse_proxy

Docker. (s. f.). *Docker Compose*. Docker Documentation. https://docs.docker.com/compose/

Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc7519.html

Microsoft. (s. f.). *TypeScript documentation*. TypeScript. https://www.typescriptlang.org/docs/

Oracle. (s. f.). *MySQL 8.0 Reference Manual*. MySQL. https://dev.mysql.com/doc/refman/8.0/en/

OpenJDK. (s. f.). *GNU General Public License, version 2, with the Classpath Exception*. https://openjdk.org/legal/gplv2+ce.html

React. (s. f.). *Learn React*. React Documentation. https://react.dev/learn

Spring. (s. f.). *Spring Boot Reference Documentation*. https://docs.spring.io/spring-boot/reference/

Spring. (s. f.). *Spring Security Reference*. https://docs.spring.io/spring-security/reference/

Tailwind Labs. (s. f.). *Styling with utility classes*. Tailwind CSS Documentation. https://tailwindcss.com/docs/styling-with-utility-classes

Vite. (s. f.). *Getting Started*. Vite Documentation. https://vite.dev/guide/

Universidad Tecnológica Nacional. (2026). *TanStack Query y el manejo de Server State* [Apunte teórico, Unidad 7]. Programación IV, Tecnicatura Universitaria en Programación a Distancia. Campus Virtual UTN.

Universidad Tecnológica Nacional. (2026). *TanStack Form + React + TypeScript* [Apunte teórico, Unidad 8]. Programación IV, Tecnicatura Universitaria en Programación a Distancia. Campus Virtual UTN.

Universidad Tecnológica Nacional. (2026). *Tailwind CSS 4: Componentes UI y manejo de estados* [Apunte teórico, Unidad 8]. Programación IV, Tecnicatura Universitaria en Programación a Distancia. Campus Virtual UTN.

Universidad Tecnológica Nacional. (2026). *Zustand en React: manejo de estado global* [Apunte teórico, Unidad 9]. Programación IV, Tecnicatura Universitaria en Programación a Distancia. Campus Virtual UTN.

Universidad Tecnológica Nacional. (2026). *Seguridad en aplicaciones modernas y APIs REST* [Apunte teórico, Unidad 9]. Programación IV, Tecnicatura Universitaria en Programación a Distancia. Campus Virtual UTN.